

PANDUAN KOMPREHENSIF REPLIKASI DATABASE HETEROGEN

Arsitektur, Prasyarat, & Setup Detil One-Way Replication Menggunakan Debezium, Apache Kafka (Oracle, PostgreSQL, dan MySQL)

1. Topologi Komponen Utama & Alur CDC

Replikasi database heterogen lintas platform memanfaatkan metode **Change Data Capture (CDC)**. Data dialirkan secara asinkron dari database Master (Source) menuju database Slave (Target/Replica) tanpa intervensi langsung pada level tabel operasional aplikasi, melainkan dengan membaca berkas log transaksi internal database secara *real-time*.

Komponen	Peran Utama	Mekanisme Kerja Internal
Source DB (Master)	Produsen Data Utama	Mencatat operasi DML ke dalam berkas log biner khusus sebelum data ditulis secara fisik ke dalam penyimpanan cakram (disk).
Debezium (Source)	Log Scanner & Parser	Bertindak selaku pembaca log engine biner spesifik secara konstan, mengemas modifikasi data ke dalam format JSON standar event CDC.
Apache Kafka	Event Stream Broker	Menyediakan pipa distribusi pesan (<i>message broker</i>) berkinerja tinggi, mengelompokkan event ke dalam wadah bernama Topic .
Kafka Connect (Sink)	Konsumen Penyedia SQL	Mengambil muatan (payload) JSON dari Kafka Topic, menyusun kembali sintaks SQL manipulasi data, dan mengeksekusinya ke Target DB.
Target DB (Slave)	Replika Read-Only	Menerima modifikasi dari Sink Connector dan menyediakannya untuk kebutuhan pembacaan beban kerja analitik/aplikasi downstream.

2. Matriks Konektor Skenario One-Way Replication

Ketika membangun replikasi satu arah, penentuan kombinasi jenis database Master dan Slave akan mendikte pustaka plugin konektor yang wajib didaftarkan pada klaster Kafka Connect:

Database Master (Source)	Database Slave (Target)	Source Connector (Debezium)	Sink Connector (Confluent/JDBC)
PostgreSQL	MySQL atau Oracle	<code>io.debezium.connector.postgresql.PostgresConnector</code>	<code>io.confluent.connect.jdbc</code>
MySQL	PostgreSQL atau Oracle	<code>io.debezium.connector.mysql.MySqlConnector</code>	<code>io.confluent.connect.jdbc</code>
Oracle	PostgreSQL atau MySQL	<code>io.debezium.connector.oracle.OracleConnector</code>	<code>io.confluent.connect.jdbc</code>

3. Pengaturan Granular: Filter Tabel & Kolom

Penggunaan Debezium mengizinkan penyaringan skema data secara presisi (Granular Filtering) guna efisiensi bandwidth jaringan maupun keamanan data:

- **Table-Level Filtering (`table.include.list`):** Membatasi pelacakan hanya pada tabel tertentu. Ditulis dalam format eksplisit: `nama_skema.nama_tabel` (Postgres/Oracle) atau `nama_database.nama_tabel` (MySQL).
- **Column-Level Filtering (`column.exclude.list`):** Mengecualikan kolom yang menyimpan muatan sensitif (seperti token, password, data finansial) agar tidak ikut dikirimkan ke dalam broker Kafka.

4. Spesifikasi Prasyarat Level Database Engine (Sisi Master)

Agar berkas log internal dapat dibaca secara aman oleh agen Debezium, masing-masing database engine yang ditunjuk sebagai **Master** wajib dikonfigurasi sebagai berikut:

A. PostgreSQL Sebagai Master

- Ubah nilai parameter pada file `postgresql.conf`: `wal_level = logical` dan tingkatkan batasan maksimum slot replikasi: `max_replication_slots = 4` (atau lebih).
- User database yang ditunjuk wajib memiliki atribut otorisasi `REPLICATION` atau bertindak sebagai `SUPERUSER`.

B. MySQL Sebagai Master

- Konfigurasi file pengaturan server MySQL (`my.cnf`) dengan parameter wajib:

```
[mysqld]
log-bin=mysql-bin
binlog_format=row
binlog_row_image=full
```

- Berikan hak istimewa minimal bagi user Debezium: `GRANT SELECT, RELOAD, SHOW DATABASES, REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'dbzuser';`

C. Oracle Sebagai Master

- Database wajib beroperasi penuh dalam mode ARCHIVELOG menggunakan fitur Oracle LogMiner.
- Aktifkan perekaman data suplemen log sekunder lewat instruksi SQL berikut:

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;  
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;
```

- User Debezium harus didaftarkan dengan hak akses khusus berupa `SELECT ANY TRANSACTION` serta hak eksekusi modul internal `DBMS_LOGMNR`.

5. Spesifikasi Prasyarat Level Database Engine (Sisi Slave/Target)

Bila database ditunjuk bertindak selaku **Slave**, beban komputasi log transaksi tidak diperlukan, namun ada beberapa restriksi skema data:

- **Skema Tabel Manual (DDL):** Pengaturan parameter `auto.create` direkomendasikan diset `false`. Struktur tabel tujuan disarankan telah dibangun secara manual terlebih dahulu sebelum aliran data diaktifkan, khususnya pada Oracle yang memiliki struktur tipe data ketat (seperti `NUMBER` atau `VARCHAR2`).
- **Kolom Kosong (Nullable):** Apabila pada konfigurasi Master terdapat proses pengecualian kolom sensitif (`Column Exclude`), maka pada skema database Slave, kolom bersangkutan wajib diatur agar menerima nilai kosong (`NULL`) atau memiliki nilai bawaan standar (`DEFAULT VALUE`) agar tidak memicu kegagalan eksekusi SQL oleh Sink Connector.

6. Berkas Template Konfigurasi JSON Terpadu (REST API Connect)

A. Contoh Registrasi Konektor Master (MySQL Source)

```
{  
  "name": "mysql-source-connector",  
  "config": {  
    "connector.class": "io.debezium.connector.mysql.MySqlConnector",  
    "tasks.max": "1",  
    "database.hostname": "mysql_master_host",  
    "database.port": "3306",  
    "database.user": "dbzuser",  
    "database.password": "dbzpassword",  
    "database.id": "184054",  
    "topic.prefix": "cdc_mysql",  
  
    "table.include.list": "production_db.users,production_db.orders",  
    "column.exclude.list": "production_db.users.password",  
  
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",  
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",  
    "key.converter.schemas.enable": "false",  
    "value.converter.schemas.enable": "false"  
  }  
}
```

B. Contoh Registrasi Konektor Slave (Oracle Sink Target)

```
{
  "name": "oracle-sink-connector",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "tasks.max": "1",
    "topics": "cdc_mysql.production_db.users,cdc_mysql.production_db.orders",
    "connection.url": "jdbc:oracle:thin:@oracle_slave_host:1521/ORCLPDB1",
    "connection.user": "REPLICA_USER",
    "connection.password": "replica_password",

    "insert.mode": "upsert",
    "pk.mode": "record_key",
    "pk.fields": "id",
    "auto.create": "false",
    "table.name.format": "${topic.table}", /* Mengarahkan penulisan nama tabel dinamis */

    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}
```

7. Tantangan Teknis Integrasi Multi-Platform

A. Sinkronisasi Zona Waktu (Timezone): Ketidakcocokan format penyimpanan tipe data Temporal/ Timestamp dapat menyebabkan pergeseran waktu beberapa jam. Disarankan menyamakan konfigurasi sistem operasi container (Docker env) seluruh database ke dalam satu zona waktu seragam (misal Asia/Jakarta atau UTC).

B. Sensitivitas Huruf (Case-Sensitivity): Oracle Database secara baku menyimpan tabel/kolom dalam format kapital besar (UPPERCASE). Ketika menerima data dari MySQL atau Postgres yang bersifat huruf kecil, gunakan transformasi internal Kafka Connect atau siapkan tabel Oracle menggunakan tanda kutip ganda jika ingin mempertahankan case huruf.